

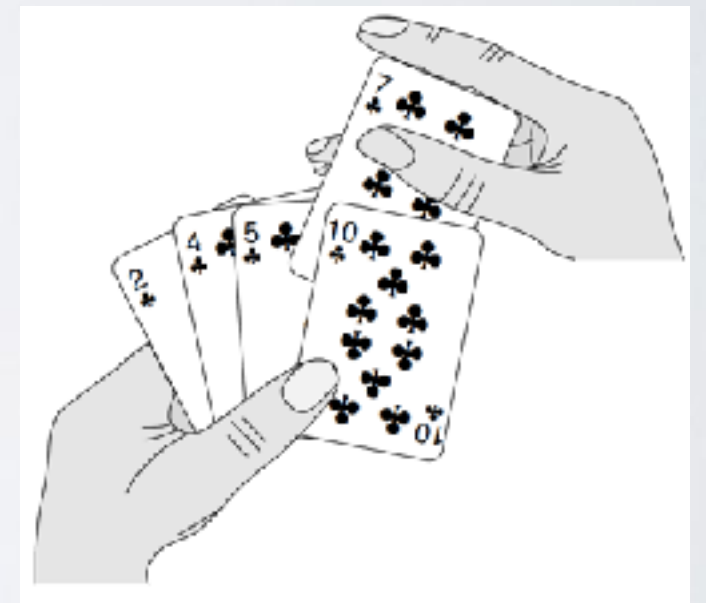
COMPLEXITÉ

Cheikh KACFAH

cheikh.kacfah@institutssaintjean.org

EXEMPLE I : TRI PAR INSERTION / PRINCIPE

- Tri très efficace, pour le tri de données de petites tailles
- S'inspire de la manière dont les gens tiennent des cartes à jouer.
 - Au début, la main gauche du joueur est vide et ses cartes sont posées sur la table. Il prend alors sur la table les cartes, une par une, pour les placer dans sa main gauche.
 - Pour savoir où placer une carte dans son jeu, le joueur la compare avec chacune des cartes déjà présentes dans sa main gauche, en examinant les cartes de la droite vers la gauche
 - A tout moment, les cartes tenues par la main gauche sont triées ; ces cartes étaient, à l'origine, les cartes situées au sommet de la pile sur la table.



© Introduction à l'algorithmique

EXEMPLE 1: TRI PAR INSERTION / ALGO

TriInsertion(T)

1. pour $j = 2$ à $\text{longueur}[T]$ faire

2. **courant** = $T[j]$;

3. $i = j - 1$;

Insertion de l'élément courant dans la séquence triée $T[1\dots j-1]$

4. tant que $i > 0$ ET $T[i] > \text{courant}$ faire

5. $T[i + 1] = T[i]$; #Décalage des éléments vers la droite

6. $i \leftarrow i - 1$; #Fin Tant que

7. $T[i+1] = \text{courant}$; #On a trouvé la position de l'élément courant | Fin Pour

EXEMPLE II: TRI FUSION / PRINCIPE

- L'algorithme du tri par fusion suit fidèlement la méthodologie diviser-pour-régner i.e. diviser un problème en sous problème et les résoudre de manière récursive
- Intuitivement, il agit de la manière suivante :
 - **Diviser** : Diviser la suite de **n** éléments à trier en deux sous-suites de **n/2** éléments chacune.
 - **Régner** : Trier les deux sous-suites de manière récursive en utilisant le tri par fusion.
 - **Combiner** : Fusionner les deux sous-suites triées pour produire la réponse triée.

EXEMPLE II: TRI FUSION / ALGO

Fusion(T,p,q,r)

1. $n1 = q - p + 1$;
2. $n2 = r - q$;
3. créer tableaux $L[1 \dots n1 + 1]$ et $R[1 \dots n2 + 1]$
4. **pour** $i = 1$ **à** $n1$ **faire**
5. $L[i] \leftarrow T[p + i - 1]$;
6. **pour** $j = 1$ **à** $n2$ **faire**
7. $R[j] \leftarrow T[q + j]$;
8. $L[n1 + 1] = \infty$;
9. $R[n2 + 1] = \infty$;
10. $i = 1$;
11. $j = 1$;
12. **pour** $k = p$ **à** r **faire**
13. **Si** $L[i] \leq R[j]$ **Alors** # Lequel des éléments actuels du tableau droit ou gauche est le plus petit?
14. $T[k] = L[i]$; # L'élément du tableau gauche est le plus petit
15. $i = i + 1$;
16. **Sinon**
17. $T[k] = R[j]$; # L'élément du tableau droit est le plus petit
18. $j = j + 1$;

COMPLEXITÉ: DÉFINITION

- **Complexité:**
 - Nombre d'opérations élémentaires qu'il doit effectuer pour mener à bien un calcul en fonction de la taille des données d'entrée.
 - Permet de répondre à la question: **Comment varie le temps de calcul en fonction de la taille des données?**
- Focus:
 - Taille des données
 - Temps de calcul

COMPLEXITÉ:TAILLE DES DONNÉES

- La taille des données (ou des entrées) va dépendre du **codage** de ces entrées. On choisit comme taille la ou les dimensions les plus **significatives et pertinentes pour le problème**. Cela peut être:
 - des éléments (peu importe le type) : le nombre d'éléments ;
 - des nombres : nombre de bits nécessaires à la représentation de ceux-là ;
 - des polynômes : le degré, le nombre de coefficients non nuls ;
 - des matrices **$m \times n$** : **$\max(m, n)$, $m.n$, $m+n$** ;
 - des graphes : nombre de sommets, nombre d'arcs, produit des deux ;
 - des listes, tableaux, fichiers : nombre de cases, d'éléments ;
 - des mots : leur longueur (moyenne, maximale).

COMPLEXITÉ: TEMPS DE CALCUL

- Le temps de calcul d'un **programme** dépend de la quantité de données, mais aussi:
 - de leur encodage ;
 - de la qualité du code engendré par le compilateur ;
 - de la nature et la rapidité des instructions du langage ;
 - de la qualité de la programmation ;
 - de l'efficacité de l'algorithme.

COMPLEXITÉ: TEMPS DE CALCUL II

- Cependant, nous cherchons à calculer la complexité de l'algorithme qui elle ne dépend ni de l'ordinateur, ni du langage utilisé, ni du programmeur, ni de l'implémentation.
 - de la nature et la rapidité des instructions du langage ;
 - de la qualité de la programmation ;
 - de l'efficacité de l'algorithme.
- On travaille supposément sur une RAM (Random Access Machine) à processeurs uniques:
 - Instructions exécutées l'une après l'autre, sans opérations simultanées
 - Mémoire infinie
 - Accès à la mémoire en temps constant

COMPLEXITÉ: OPÉRATIONS FONDAMENTALES

- Nous nous intéressons au nombre d'opérations fondamentales effectuées dans le programme
- Mais dire d'une opération qu'elle est fondamentale dépend du problème à traiter

Problèmes	Opérations fondamentales
Recherche d'un élément dans une liste	Comparaisons
Tri d'une liste, d'un fichier, etc.	Comparaisons, déplacements
Multiplication des matrices réelles	Multiplications et additions
Addition des entiers binaires	Opérations binaires

COMPLEXITÉ: COÛTS DES OPÉRATIONS / COÛTS DE BASE

Pour la complexité en temps, plusieurs choix sont possibles:

1. Calculer (en fonction de la taille des données d'entrée) le **nombre d'opérations élémentaires** (addition, comparaison, affectation, ...) requises par l'exécution puis le multiplier par le **temps moyen de chacune d'elle**
2. Pour un algorithme avec essentiellement des calculs numériques, compter les **opérations coûteuses** (multiplications, racine, exponentielle, ...)
3. Sinon, compter le nombre d'appels à l'opération la plus fréquente (cf. tableau précédent).

Ici, nous adoptons le point de vue suivant : **l'exécution de chaque ligne de pseudo code demande un temps constant**. Nous noterons **c_i** le coût en temps de la ligne **i** .

COMPLEXITÉ: COÛT D'UN ALGORITHME SÉQUENTIEL

1. **Séquence**: $T_{\text{séquence}} = \text{Somme } (T_{\text{éléments_de_la_séquence}})$
2. **Alternative** (Si C alors J sinon K): $T_{\text{alt}} = T_C + \max(T_J, T_K)$
3. **Itération bornée** (pour i de j à k faire B):
$$T_{\text{iterationBornée}} = (k-j+1) (T_{\text{entête}} + T_B) + T_{\text{entête}}$$
4. **Itération non bornée** (tant que C faire B):
$$T_{\text{iterationNonBornée}} = N_{\text{boucles}} (T_C + T_B) + T_C$$
5. **Répéter B jusqu'à C** :
$$T_{\text{répéter}} = N_{\text{boucles}} (T_C + T_B)$$

EXEMPLE 1: FACTORIEL / COMPLEXITÉ

Factoriel(n)

1. `factoriel = 1 ;`
 2. `Si n >= 2 Alors`
 3. `Pour i = 2 à n faire`
 4. `factoriel = factoriel * i ;`
 5. `FinPour`
 6. `Retourner factoriel ;`
1. Le coût de la **ligne 1** est **c1**
 2. En **ligne 2** on a un **Si A alors B**. Ceci est équivalent à **Si A alors B sinon C** avec un **sinon** « vide ». Son coût est **coût_A + max(coût_B, coût_C)** soit **coût_A + coût_B** car **C** a un coût nul dans ce cas. Notons que **coût_A = c2**.
 1. **coût_B** est celui de la boucle **Pour** et il vaut **N_{boucles} × (T_{entête} + T_{instrBoucles}) + T_{entête}**. Notons **T_{entête} = c3** et **T_{instrBoucles} = c4**
 2. Le coût du bloc des lignes 2 à 5 est donc **c2 + (n-1) × (c3 + c4) + c3**
 3. Le temps de notre algorithme est donc **c1 + c2 + (n-1) × (c3 + c4) + c3** soit de la forme **n.k1 + k2**

EXEMPLE II: TRI INSERTION / COMPLEXITÉ

TriInsertion(T)

1. Pour $j = 2$ à $\text{longueur}[T]$ faire
2. **courant** = $T[j]$;
3. $i = j - 1$;
4. TantQue $i > 0$ ET $T[i] > \text{courant}$ faire
5. $T[i + 1] = T[i]$;
6. $i \leftarrow i - 1$;
7. FinTantQue
8. $T[i+1] = \text{courant}$;
9. FinPour

1. Le coût de la boucle **Pour** vaut $N_{\text{boucles}} \times (T_{\text{entête}} + T_{\text{instrBoucles}}) + T_{\text{entête}}$. Avec $N_{\text{boucles}} = n - 1$ et $T_{\text{entête}} = c1$ et $T_{\text{instrBoucles}} = c2 + c3 + T_{\text{TantQue}} + c8$
2. Le coût de la boucle TantQue est $N_{\text{boucles}} \times (T_{\text{entête}} + T_{\text{instrBoucles}}) + T_{\text{entête}}$. Avec $T_{\text{entête}} = c4$ et $T_{\text{instrBoucles}} = c5 + c6$. $N_{\text{boucles}} = t_j - 1$, j dans $[2, n]$ où t_j est le nombre d'itérations pour une valeur de j donnée. Au pire i ira de $j-1$ à 0 soit j itérations. On a donc $N_{\text{boucles}} = j-1$;
3. Le temps total est donc $(n-1)(c1+c2+c3+c4+c8) + c1 + [\text{Somme}(j-1)_{j=2 \text{ à } n} \times (c4+ c5 + c6)]$
 $\text{Somme}(j-1)_{j=2 \text{ à } n} = n(n-1)/2$
4. Après factorisation: $n^2.k1 + n.k2 + k3$

LES DIFFÉRENTES MESURES DE COMPLEXITÉ

Soit **A** un algorithme, **n** un entier, **D_n** l'ensemble des entrées de taille **n** et une entrée **d** $\in$ **D_n**. Posons : **coût_A(d)** le nombre d'opérations fondamentales effectuées par **A** avec l'entrée **d**.

1. Meilleur des cas: **Min_A(n)**= $\min\{\text{coût}_A(d) \mid d \in D_n\}$.
2. Cas moyen: C'est le temps d'exécution moyen ou attendu d'un algorithme.
Question: qu'est ce qu'une entrée "moyenne" pour un problème particulier?
Moy_A(n) = $\sum_{d \in D_n} p(d) \cdot \text{coût}_A(d)$ avec **p(d)** une loi de probabilité sur les entrées.
3. Pire des cas: **Max_A(n)**= $\max\{\text{coût}_A(d) \mid d \in D_n\}$.

RÉFÉRENCES

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms*. MIT press.
- Eric TRICHET. Introduction à la complexité algorithmique. Centre de Ressources et d'Innovation Pédagogiques de l'Université de Limoges